Reading Exercise 04 — The Comment Hunt

What the code cannot say about itself.

What this exercise trains

Comments are the parts of a program that resist the compiler. The compiler ignores them. They exist only for the reader, which makes them evidence: evidence of what the programmer thought needed to be said that the code could not say alone.

Most comments are noise. They describe what the code already shows: `/ increment i /`. A good comment says something the code cannot — a hidden constraint, a surprising invariant, a decision made for reasons that are not visible in the structure.

This exercise teaches you to tell the difference.

Step 1 — The taxonomy [5 minutes]

There are roughly four kinds of comments in production code:

> **What comments:** describe what the code does. Often redundant. ("Add 1 to x.")
> **Why comments:** explain a decision or constraint that isn't visible from the code. ("Must be called before fork() because...")
> **Warning comments:** flag something dangerous, counterintuitive, or easily broken. ("Don't change the order of these two lines.")
> **Historical comments:** record a bug fix, a regression, or a change made for a specific reason. ("Workaround for GCC bug #12345.")
>
> **Write down:** before you open any file, predict which kind of comment will be most common in TCC. Why?

Step 2 — The hunt in tccgen.c [20 minutes]

Open `practice/tcc/tccgen.c`. This is TCC's largest and most complex file.

Your task: find the five most interesting comments in this file. "Interesting" means: a comment that says something the code itself does not say. A comment that would change how you understand the surrounding code.

Do not read the whole file. Skim. Look for comments. Read the code around them just enough to understand why the comment exists.

```
grep -n "/\*\|//" practice/tcc/tccgen.c | grep -v "TODO\|FIXME\|XXX\|\*/" |
 head -100
```

Or just open the file and page through it.

> **Write down:** five comments, with line numbers, and one sentence explaining why each one is interesting.

Step 3 — The hunt in tcc.h [10 minutes]

Open `practice/tcc/tcc.h`. Header files often carry high-level architecture comments. Repeat the hunt: find three comments that reveal something about TCC's design.

> **Write down:** three comments, with line numbers, and one sentence each.

Step 4 — Classify [5 minutes]

Return to your eight comments. Classify each one using the four types from Step 1.

> **Write down:** the classification for each. Were any of them more than one type at once?

Step 5 — The thing comments can't fix [5 minutes]

Find one place in `tccgen.c` where you think a comment is *missing* — where the code is confusing or surprising and there is no comment to explain it.

> **Write down:** the line number and what you think the comment should say.

Step 6 — Compare with GCC [10 minutes]

In `02/gcc/gcc/` (the original GCC source), find a comment that you could not have found in TCC — something about GCC's architecture, design philosophy, or history that is only possible in a much larger and older codebase.

```
grep -rn "historical\|originally\|compat\|quirk\|FIXME\|legacy" 02/gcc/gcc/
c-parser.c | head -20
```

Write down: the comment, the file and line number, and what it tells you that the code alone would not.

Debrief

Comments are the programmer's shadow. They appear where the code casts doubt — where the structure alone is insufficient, where time has passed and the reasons are no longer obvious, where the next reader would otherwise stumble.

Reading the comments in a codebase tells you where the hard problems are. Not the places that are large or complex, but the places that are *subtle* — the places where the programmer felt compelled to say something the code could not say for itself.

In TCC, the comment density is lower than in GCC. Bellard's code tends toward the self-evident. In GCC, the comments are denser because the problems are older and more tangled. Both tell you something about their authors.

Next: RE-05 — GCC from a Distance — navigating a large codebase without reading all of it.